

AI-Augmented Backend Development Project :

TITLE : AnswerBot - AI-powered FAQ and customer support automation

Project Description

AnswerBot - AI-powered FAQ and customer support automation is a robust REST API platform for FAQ creation, authentication, search, and AI-assisted content management. Built with Node.js and Express.js, it uses MongoDB with Mongoose for data storage and validation, JWT for stateless authentication, bcrypt for password hashing, and Google Gemini for AI-assisted question answering and FAQ generation. Centralized error handling and protected routes help maintain reliable backend operations.

Scenario

Problem

Customer support teams handle repetitive questions while maintaining FAQ records manually. Organizing topics, writing clear answers, searching existing information, and controlling access can be time-consuming.

AI Solution

The backend centralizes FAQ management and adds Google Gemini assistance. Authenticated users can create/manage FAQs, search records, request AI-generated answers, and generate FAQ pairs from a topic.

Impact

Administrators, content creators, authenticated users, and public users benefit from one backend service that reduces repetitive authoring work and provides structured, searchable FAQ data.

System Requirements :

1. Software Requirements

- Windows 10/11, macOS, or Linux.
- Node.js (v18+).
- npm for dependency management.
- Express.js for REST APIs.
- MongoDB Atlas / MongoDB with Mongoose ODM.
- jsonwebtoken (JWT) for protected routes.
- bcrypt for password hashing.
- Google Gemini API / @google/genai for AI features.
- Postman or Thunder Client for API testing.
- Visual Studio Code for development.
- Git/GitHub or project storage.

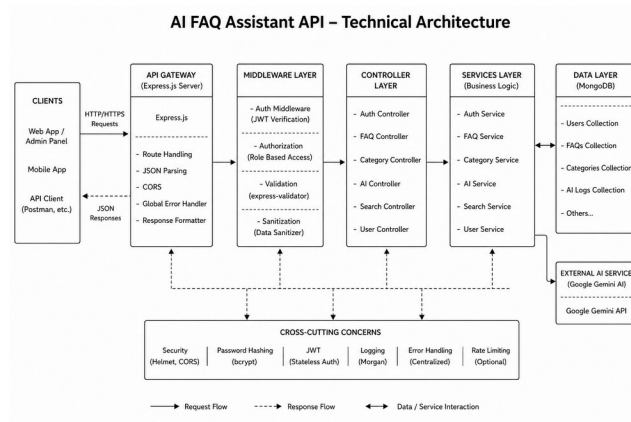
2. Hardware Requirements

- Intel Core i5 (8th Gen+) / AMD Ryzen 5 or equivalent.
- 8 GB RAM minimum; 16 GB recommended.
- At least 1 GB free project workspace.
- Internet connection for MongoDB Atlas and Gemini API.

Epic-1: Project Architecture

The AI FAQ Assistant uses a layered REST API architecture. Requests enter through Express, pass through validation/authentication where required, reach controllers/services, and interact with MongoDB through Mongoose or Google Gemini through the AI service.

Technical Architecture

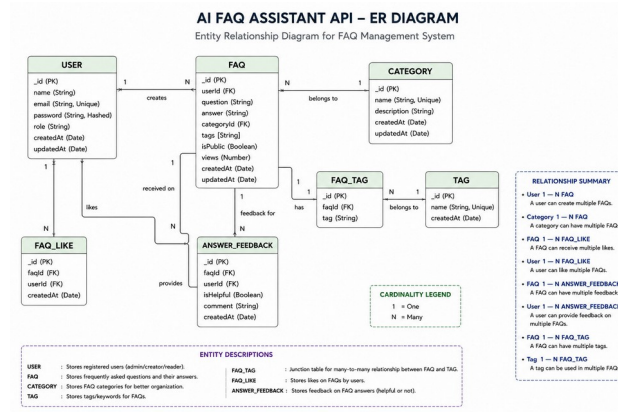


Components Definition

- **Interface:** Postman/Thunder Client sends HTTP requests and receives JSON.
- **Express Server Gateway:** routes requests, parses JSON, handles CORS, and returns responses.
- **Authentication Middleware:** verifies Bearer JWT tokens for protected routes.
- **Services:** contains authentication, FAQ, search, and AI processing logic.
- **Database Interface:** Mongoose translates operations into MongoDB queries.
- **AI Flow:** Google Gemini handles AI-assisted question answering and FAQ generation.

Entity-Relationship (ER) Diagram Description

The MongoDB data model contains user and FAQ collections with logical relationships to categories and dynamically generated AI content. FAQ records store the user who created them.



Collections

1. User Entity

- `_id` (ObjectId primary key)
- name, email, password
- role
- createdAt, updatedAt

2. FAQ Entity

- `_id` (ObjectId primary key)
- question, answer, category
- createdBy (User reference)
- createdAt, updatedAt

3. AI Generation Entity (Logical)

- topic
- generatedQuestion
- generatedAnswer
- generatedCategory
- generatedAt

4. Category Entity (Logical)

- categoryName
- description
- totalFAQs

Key Structural Mappings

- User → FAQ (One-to-Many): a user can create/manage multiple FAQ records.
- Category → FAQ (One-to-Many): a category can contain multiple FAQ records.
- User → AI Generated FAQ (One-to-Many): a user can generate multiple AI-assisted FAQs.
- FAQ → Search Results: search requests retrieve matching FAQ documents.

Key Features

- JWT Authentication & Authorization
- AI FAQ Generation using Google Gemini
- AI-assisted question answering
- FAQ keyword search
- Create, Read, Update, Delete FAQ operations
- Category organization
- bcrypt password hashing
- Centralized error handling
- Mongoose schema validation
- RESTful Express API
- MongoDB integration

Roles and Responsibilities

Admin

- Manage users.
- View, update, and delete FAQs.
- Monitor AI-generated content.
- Manage categories.
- Access complete functionality.

Content Creator

- Create, edit, and delete owned FAQs.
- Generate FAQs using AI.
- Organize FAQs under categories.

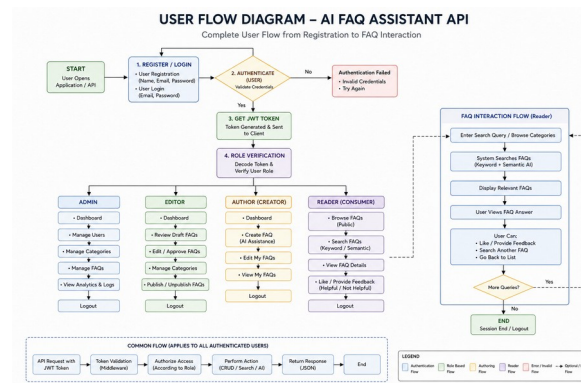
Authenticated User

- View/search FAQs.
- Generate AI-assisted answers.
- Access personal profile information.

Public User

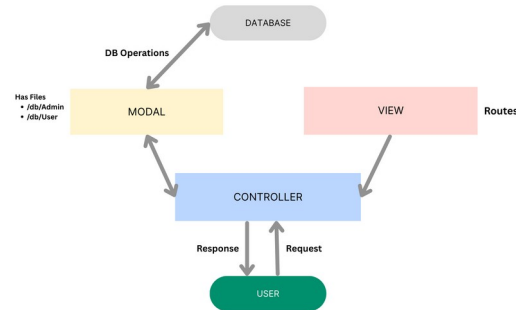
- View/search published FAQs.
- Cannot modify FAQ records.
- Cannot access protected endpoints.

User Flow



MVC Architecture Pattern

The system follows the Model-View-Controller pattern to separate data definitions, execution logic, and API routing. As a headless backend, the View layer is represented by the REST API routing layer rather than HTML templates.



Model Layer

Mongoose schemas define fields, validation rules, defaults, and references for MongoDB resources.

Controller Layer

Controllers receive route requests, coordinate database or AI services, and return standardized JSON responses.

View Layer (Routing Network Interface)

API routes map HTTP requests to their associated controller handlers.

Advantages of MVC

- Separation of concerns
- Maintainability
- Testing
- Reusable components
- Team collaboration
- Scalability

Epic 2: Project Setup and Configuration

Creating Project Folder

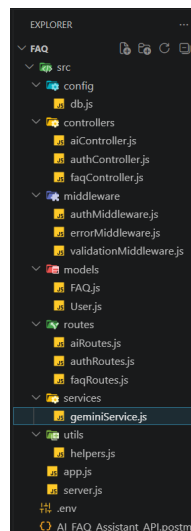
1. Create a new folder with the project name.
2. Create the src folder and backend source directories.
3. Open the folder in VS Code.

Server Setup (npm init)

npm init -y

npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai

File and Folder Structure Matrix



The source is organized into server startup, Express setup, models, controllers, routes, middleware, and services.

Epic 3: Backend Development

Core System Script Implementations

1. Gateway Server Node (src/server.js)

Loads environment variables, connects to MongoDB, creates the server, and listens on the configured port.

```

1 // src/server.js
2 const app = require('./app');
3 const connectDB = require('./config/db');
4 const mongoose = require('mongoose');
5 const dotenv = require('dotenv');
6 const { errorHandler } = require('./middleware/errorMiddleware');
7
8 // Load environment variables
9 dotenv.config();
10
11 // Connect to database
12 connectDB();
13
14 // Create server
15 const server = app.listen(PORT, () => {
16   console.log(`Server running in ${process.env.NODE_ENV} mode on port ${PORT}`);
17 });
18
19 // Handle unhandled promise rejections
20 process.on('unhandledRejection', (err, promise) => {
21   console.error('Unhandled Rejection: ', err.message);
22   // Close server & exit process
23   server.close(() => process.exit(1));
24 });
25
26 // Start server
27 startServer();
  
```

2. Gateway Express Setup (src/app.js)

Configures Express middleware, JSON parsing, CORS, route registration, and error handling.

```

1 // src/app.js
2 const express = require('express');
3 const cors = require('cors');
4 const dotenv = require('dotenv');
5 const { errorHandler } = require('./middleware/errorMiddleware');
6
7 // Load environment variables
8 dotenv.config();
9
10 const app = express();
11
12 // Enable CORS
13 app.use(cors());
14
15 // Body parser middleware
16 app.use(express.json());
17 app.use(express.urlencoded({ extended: false }));
18
19 // Basic status check route
20 app.get('/', (req, res) => {
21   res.json({
22     success: true,
23     message: 'AI Assistant API is running.',
24     version: '1.0.0',
25   });
26 });
27
28 // Register API Routes
29 app.use('/api/auth', require('./routes/authRoutes'));
30 app.use('/api/users', require('./routes/usersRoutes'));
31 app.use('/api/ai', require('./routes/aiRoutes'));
32
33 // Handle socket.io polling requests gracefully (if client attempts websocket connection)
34 app.use('/socket.io', (req, res) => {
35   res.status(404).json({ error: 'Socket.io not enabled' });
36 });
  
```

3. Security Access Middleware (src/middleware/authMiddleware.js)

Extracts and verifies the Bearer JWT and attaches authenticated user context.

```

1 // src/middleware/authMiddleware.js
2 const jwt = require('jsonwebtoken');
3 const User = require('../models/user');
4
5 const protect = async (req, res, next) => {
6   let token;
7
8   if (req.headers.authorization && req.headers.authorization.startsWith('bearer ')) {
9     token = req.headers.authorization.split(' ')[1];
10
11     try {
12       // Verify token
13       const decoded = jwt.verify(token, process.env.JWT_SECRET || 'your_jwt_secret_key_here_must_be_in');
14
15       // Get user from the token, exclude password
16       req.user = await User.findById(decoded.id).select('-password');
17
18       if (!req.user) {
19         return res.status(401).json({
20           success: false,
21           message: 'Not authorized, user not found',
22         });
23       }
24     } catch (error) {
25       console.error(error);
26       return res.status(401).json({
27         success: false,
28         message: 'Not authorized, token failed',
29       });
30     }
31   }
32
33   next();
34 }
  
```

4. Centralized Error Handler Middleware (src/middleware/errorMiddleware.js)

Provides consistent error responses for validation, authentication, and server failures.

```

1 // src/middleware/errorMiddleware.js
2 const { validationResult } = require('express-validator');
3
4 const errorHandler = (err, req, res, next) => {
5   // Development: Log the error and show it to the client
6   console.log('Error: ', err);
7
8   // Production: Log the error and show a generic message to the client
9   const message = process.env.NODE_ENV === 'production' ? 'Internal Server Error' : err.message;
10
11   // Handle specific error types
12   if (err.name === 'ValidationError') {
13     const errors = validationResult(req);
14     return res.status(400).json({ success: false, message: errors.array().map(e => e.message) });
15   }
16
17   if (err.name === 'CastError') {
18     return res.status(400).json({ success: false, message: 'Invalid data type' });
19   }
20
21   // Handle other errors
22   return res.status(500).json({ success: false, message });
23 };
  
```

Epic 4: Database Configuration

Mongoose Schema Definitions

1. Identity Resource Document Schema (src/models/User.js)

Defines required identity fields, unique email validation, role defaults, and password hashing.

```

1 const mongoose = require('mongoose');
2 const bcrypt = require('bcrypt');
3
4 const userSchema = new mongoose.Schema(
5   {
6     name: {
7       type: String,
8       required: [true, 'Name is required'],
9       trim: true,
10    },
11    email: {
12      type: String,
13      required: [true, 'Email is required'],
14      unique: true,
15      lowercase: true,
16      trim: true,
17      match: [/^\w+@(\w+)\.(\w+)$/, 'Please provide a valid email address'],
18    },
19    password: {
20      type: String,
21      required: [true, 'Password is required'],
22      minlength: [6, 'Password must be at least 6 characters long'],
23    },
24  },
25   {
26     timestamps: true,
27   }
28 );
29
30 // Pre save hook to hash the password before saving
31 userSchema.pre('save', async function () {
32   if (!this.isModified('password')) {
33     return;
34   }
35   const salt = await bcrypt.genSalt(10);
36   this.password = await bcrypt.hash(this.password, salt);
37 });
  
```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Defines question, answer, category, creator reference, and timestamps.

```

1 const mongoose = require('mongoose');
2
3 const faqSchema = new mongoose.Schema(
4   {
5     question: {
6       type: String,
7       required: [true, 'Question is required'],
8       trim: true,
9     },
10    answer: {
11      type: String,
12      required: [true, 'Answer is required'],
13      trim: true,
14    },
15    category: {
16      type: String,
17      required: [true, 'Category is required'],
18    },
19    values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
20    message: ['(VALUE) is not a valid category. Choose from technology, education, health, banking, general'],
21    default: 'General',
22  },
23   {
24     timestamps: true,
25   }
26 );
27
28 // Create a reference to the user schema
29 const User = mongoose.model('User', userSchema);
30
31 // Create a reference to the faq schema
32 const FAQ = mongoose.model('FAQ', faqSchema);
  
```

Database Development

- MongoDB Atlas is used for the current database configuration.
- Mongoose provides schema validation and database access.
- FAQ records reference the user who created them.

API Router Interface Endpoints

1. Public Authentication Operations (src/routes/authRoutes.js)

Registration and login are public endpoints; login provides authentication data for protected operations.

```

1 // src/routes/authRoutes.js
2 const express = require('express');
3 const router = express.Router();
4
5 // Public routes
6 router.post('/register', validateRegister, registerUser);
7 router.post('/login', validateLogin, loginUser);
8
9 // Protected routes
10 router.get('/profile', protect, getUserProfile);
11
12 module.exports = router;
  
```

2. Protected FAQ Operations (src/routes/faqRoutes.js)

FAQ creation, update, and deletion require JWT authentication; search is exposed through the public search route.

```

1 // src/routes/faqRoutes.js
2 const express = require('express');
3 const router = express.Router();
4
5 // Public routes - GET operations
6 router.get('/search', searchFAQs); // Note: defined before /id to prevent routing issues
7 router.get('/id', getFAQById);
8
9 // Protected routes - Modification operations
10 router.post('/', protect, validateFAQ, createFAQ);
11 router.put('/:id', protect, updateFAQ);
12 router.delete('/:id', protect, deleteFAQ);
13
14 module.exports = router;
  
```

3. AI Automation Operations (src/routes/aiRoutes.js)

AI routes provide authenticated AI-assisted answer generation and FAQ generation through Google Gemini.

```

1 // src/routes/aiRoutes.js
2 const express = require('express');
3 const router = express.Router();
4
5 // Public routes
6 router.post('/generate-faq', protect, validateFAQ, generateFAQ);
7 router.post('/generate-answer', protect, validateFAQ, generateAnswer);
8
9 // Protected routes
10 router.get('/generate-faq', protect, validateFAQ, generateFAQ);
11
12 module.exports = router;
  
```

Epic 5: Project Executions

Running the API Application

- Install Postman or Thunder Client.
- Run npm install.
- Configure PORT, MONGO_URI, JWT_SECRET, and GEMINI_API_KEY.
- Start with npm run dev or npm start.

DEMO LINK:

https://drive.google.com/file/d/1S0gndPg90ky_Njz_4AAupWtW9X1pO99h/view?usp=sharing